Day 134: A GAN From Scratch: Generator and Discriminator, a Real Minimax Training Loop

MD Mutasim Billah | 52-Week Data Science to ML/AI Roadmap | Week 20, Day 2 | Complete study guide

Watch: Ian Goodfellow: Generative Adversarial Networks (NIPS 2016 tutorial) | Generative Adversarial Networks (GANs) - Explained (DataMListic)

Script: `day134_gan_from_scratch.py` · every number in this guide is copied by program from `day134_run_output.txt`, the real output of that script.

Two networks, trained against each other: a generator that turns noise into data, and a discriminator that guesses whether a sample is real. Both are written from scratch on Day 126's network parts. The generator's gradient, which flows back through the discriminator, matches finite differences to 4.2e-07; a trained discriminator lands within 0.0562 of the formula for the best possible one. On a 1-D two-peaked data set the generator finds both peaks, though two of five runs sat on one peak for 1,000 to 3,000 steps first. On MNIST the first setting failed: with Adam's usual momentum (0.9) the generator never made some digits at all, and its commonest digit took 0.630 of its samples. With momentum 0.5 every digit appears, the commonest takes 0.177, and the samples are not copies of training images.

0. Where today fits

DEFINITION — start here, no prior knowledge assumed

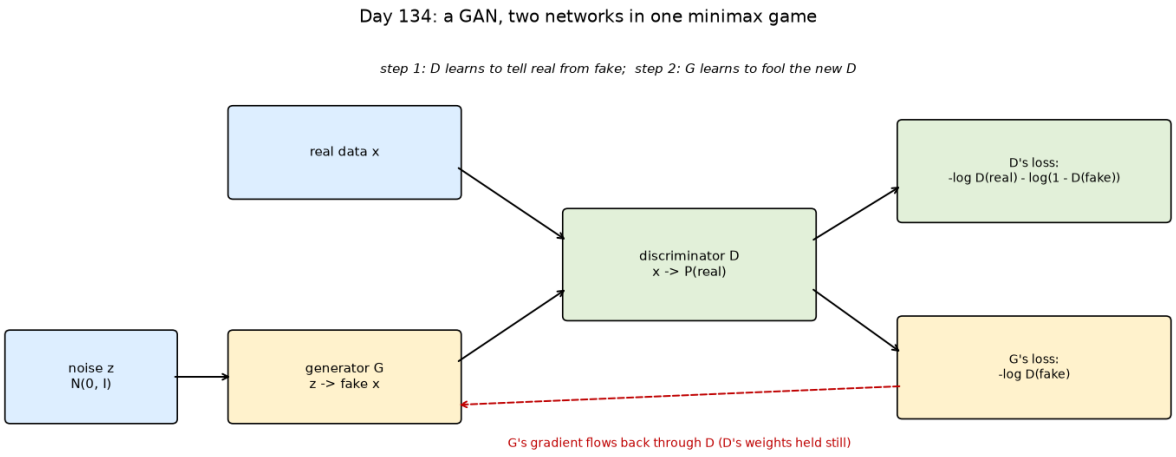
A **generative adversarial network** (GAN) has two players. The **generator** G turns random noise z into a sample G(z). The **discriminator** D looks at a sample and outputs D(x), its probability that x is real. D is trained to tell real from fake; G is trained to make D call its fakes real.

Day 133's Gaussian models wrote p(x) down as a formula. A GAN never does: the only thing that tells G what real data looks like is the gradient of D's verdict. That makes GANs able to model complicated data, and hard to train, which is Day 135's subject.

REAL-WORLD SCENARIO

A forger and an art inspector. The inspector studies real paintings and the forger's latest fakes; the forger improves by learning which details gave the fakes away. Neither ever writes down what a real painting is.

SYSTEM DIAGRAM



The two players, and where each gradient goes: D's from its own loss; G's back through D, with D's weights held still.

Today's part for the week's build

`day134_exercise.py` is the GAN's parts: Adam with a momentum setting, both losses, the generator, its gradient through the discriminator, and one round of the game, checked by hand. The last two lines are a real finding: a small pair of networks matched only the mean of N(3, 0.5), because its discriminator went flat (D near 0.5 at 1, 3 and 7).

ACTUAL OUTPUT

```
at D = 0.5: discriminator loss 1.386294 (2 log 2 = 1.386294); generator loss 0.693147 (log 2 =
    0.693147)
softplus(1000) = 1000.0; softplus(-1000) = 0.0 (no overflow)
d loss / d G[0][0][0, 0]: backward 0.07097488; finite difference 0.07097488
G [2, 16, 1], D [1, 16, 1], 3,000 steps on N(3, 0.5): generated mean 3.023, spread 2.292; D at x =
    1, 3, 7: 0.50 0.49 0.49
G [2, 32, 32, 1], D [1, 32, 32, 1], 3,000 steps on N(3, 0.5): generated mean 3.090, spread 0.526;
    D at x = 1, 3, 7: 0.47 0.51 0.52
```

Data: a 1-D mixture with a known density (half the samples near –2, half near +2, spread 0.5), and MNIST (cached by Day 126). Day 126's networks and Adam, Day 116's softmax regression (the judge of generated digits) and Day 119's squared distances are imported unchanged.

How to read each section

Step	What it gives you
DEFINITION	What the idea is, in plain words. Start here.
REAL-WORLD SCENARIO	An everyday situation where the same idea shows up.
MATHEMATICS	The formula, when the section introduces one.
SPECIFICATION	The exact setup of the experiment.
IMPLEMENTATION	The real code from the script.
ACTUAL OUTPUT	What the script really printed, copied unchanged.
What it means	How to read the numbers, and what they do and do not show.
Why choose this? / Why NOT this?	When the idea is the right tool, and what it costs.

1. By hand: the losses when the discriminator cannot tell

DEFINITION — start here, no prior knowledge assumed

D outputs a **logit** l and D = sigmoid(l). D's loss is –log D(real) – log(1 – D(fake)). The original paper's generator loss is log(1 – D(fake)), to be minimised (the **minimax** form); in practice G minimises –log D(fake) instead (the **non-saturating** form). Both push D(fake) up.

REAL-WORLD SCENARIO

A referee who flips a coin for every call: both teams' complaints are exactly the same size.

MATHEMATICS

$$V(D, G) = E_x[\log D(x)] + E_z[\log(1 - D(G(z)))], \quad \min_G \max_D V$$

$$d/dl\ [-\log \text{sigmoid}(l)] = \text{sigmoid}(l) - 1, \quad d/dl\ [\log(1 - \text{sigmoid}(l))] = -\text{sigmoid}(l)$$

IMPLEMENTATION

```
def softplus(Z):
    return np.maximum(Z, 0) + np.log1p(np.exp(-np.abs(Z)))
def d_loss(lr, lf):
    loss = softplus(-lr).mean() + softplus(lf).mean()
    return float(loss), (d126.sigmoid(lr) - 1) / len(lr), d126.sigmoid(lf) / len(lf)
def g_loss(lf, kind="non-saturating"):
    if kind == "non-saturating":
        return float(softplus(-lf).mean()), (d126.sigmoid(lf) - 1) / len(lf)
    return float(-softplus(lf).mean()), -d126.sigmoid(lf) / len(lf)
```

ACTUAL OUTPUT

```
D = sigmoid(0) = 0.5000 on every real and every fake sample
discriminator loss -log D(real) - log(1 - D(fake)) = 1.3863; by hand 2 log 2 = 1.3863
generator loss, non-saturating -log D(fake) = 0.6931; by hand log 2 = 0.6931
generator loss, minimax log(1 - D(fake)) = -0.6931; by hand -log 2 = -0.6931
when the discriminator is sure a fake is fake (logit -6, D(fake) = 0.0025), the gradient each fake sends back
    to the generator:
   non-saturating  d loss / d logit = -0.9975
   minimax         d loss / d logit = -0.0025
the minimax form gives almost no gradient exactly when the generator is losing; the non-saturating form gives
    almost the most
Actual output from this lesson's run.
```

What it means

At D = 0.5 everything matches the hand calculation: D's loss is 2 log 2 = 1.3863, the non-saturating loss log 2 = 0.6931, the minimax loss –log 2 = -0.6931. The second half is the reason for the non-saturating form: when D is sure a fake is fake (D(fake) = 0.0025), the minimax form sends back -0.0025 per fake, the non-saturating form -0.9975, about 400 times more, exactly when the generator most needs to learn.

WHY CHOOSE THIS?

The non-saturating loss keeps the generator learning while it is losing.

WHY NOT THIS? (TRADEOFFS)

It no longer minimises the exact quantity the theory in section 3 describes.

2. Correctness: the generator's gradient, through the discriminator

DEFINITION — start here, no prior knowledge assumed

G's loss depends on its weights only through D: noise → G → fake → D → logit → loss. So G's gradient is computed by running Day 126's backward pass through D to its *input* (D's own weight gradients are thrown away), then through G's output sigmoid, then through G. A finite-difference check nudges each of G's weights and compares.

REAL-WORLD SCENARIO

A student who learns only from a teacher's red marks: every correction comes through the teacher, but it is the student who changes.

MATHEMATICS

$$\partial L_G/\partial w \approx [L_G(w + h) - L_G(w - h)] / 2h, \quad h = 10^{-6}; \text{ relative error} = |num - ana| / (|num| + |ana|)$$

IMPLEMENTATION

```
def g_grads(G, D, Z, out="sigmoid", kind="non-saturating"):
    dacts = ["relu"] * (len(D) - 1) + ["linear"]; gacts = ["relu"] * (len(G) - 1) + ["linear"]
    F, AG = generate(G, Z, out); Af = d126.forward(D, F, dacts); lg, gf = g_loss(Af[-1], kind)
    _, dF = d126.backward(D, Af, dacts, gf)
    dL = dF * F * (1 - F) if out == "sigmoid" else dF
    return lg, d126.backward(G, AG, gacts, dL)[0], Af
```

ACTUAL OUTPUT

```
   generator output linear , non-saturating loss: largest relative error over 42 weights 2.2e-08
   generator output sigmoid, non-saturating loss: largest relative error over 42 weights 2.7e-07
   generator output sigmoid, minimax        loss: largest relative error over 42 weights 4.2e-07
the discriminator's weights were held fixed: the generator learns only through the discriminator's
    verdict
Actual output from this lesson's run.
```

What it means

All 42 weights agree with finite differences in all three settings, the worst at 4.2e-07 (float64 networks). The check runs the full pipeline every trained network uses: noise, generator, sigmoid, discriminator, loss.

WHY CHOOSE THIS?

One backward pass gives the generator its whole gradient.

WHY NOT THIS? (TRADEOFFS)

The generator can only learn what the discriminator's gradient happens to point at.

3. The optimal discriminator, and what the game minimises

DEFINITION — start here, no prior knowledge assumed

For a fixed generator, the best possible discriminator is $D^*(x) = p_{data}(x) / (p_{data}(x) + p_g(x))$. Putting D^* into the game's value gives $-\log 4 + 2\,\mathrm{JSD}(p_{data}, p_g)$, where JSD is the Jensen–Shannon divergence, a distance between distributions that is zero only when they are equal. So a generator facing a perfect discriminator is pushed to make $p_g = p_{data}$. Here the generator is fixed as N(0, 1), both densities are known exactly, and a discriminator is trained alone for 4,000 steps.

REAL-WORLD SCENARIO

An inspector who has seen how often each kind of painting comes from the gallery and from the forger: at each painting, the best guess is the gallery's share.

MATHEMATICS

$$D^*(x) = p_d(x) / (p_d(x) + p_g(x)), \quad V(D^*, G) = -\log 4 + 2\,\mathrm{JSD}(p_d, p_g)$$

$$\mathrm{JSD}(p, q) = \tfrac{1}{2}\,\mathrm{KL}(p \,||\, m) + \tfrac{1}{2}\,\mathrm{KL}(q \,||\, m), \quad m = \tfrac{1}{2}(p + q)$$

IMPLEMENTATION

```
x = np.linspace(-4, 4, 801)
d_star = p_real1d(x) / (p_real1d(x) + p_g(x)); d_net = d126.sigmoid(d126.forward(D, x[:,
    None].astype(np.float32), acts)[-1])[:, 0]
busy = (p_real1d(x) + p_g(x)) > 0.05
print(f"after 4,000 steps against N(0, 1): largest |D - D*| where either density is above 0.05:
    {np.abs(d_net - d_star)[busy].max():.4f}; "
      f"average {np.abs(d_net - d_star)[busy].mean():.4f}")
print("   x:      " + "  ".join(f"{v:6.1f}" for v in (-3, -2, -1, 0, 1, 2, 3)))
pick = [np.argmin(np.abs(x - v)) for v in (-3, -2, -1, 0, 1, 2, 3)]
print("   D*(x):  " + "  ".join(f"{d_star[i]:6.3f}" for i in pick))
print("   D(x):   " + "  ".join(f"{d_net[i]:6.3f}" for i in pick))
xs = np.linspace(-12, 12, 200001); dx = xs[1] - xs[0]; pd, pg = p_real1d(xs), p_g(xs); m = 0.5 *
    (pd + pg)
kl = lambda p, q: float(np.sum(p * np.log((p + 1e-300) / (q + 1e-300))) * dx)
jsd = 0.5 * kl(pd, m) + 0.5 * kl(pg, m)
v_star = float(np.sum(pd * np.log(pd / (pd + pg) + 1e-300) + pg * np.log(pg / (pd + pg) + 1e-300))
    * dx)
Xr, Xf = real1d(200000, np.random.RandomState(4)),
    np.random.RandomState(5).standard_normal((200000, 1)).astype(np.float32)
v_net = -d_loss(d126.forward(D, Xr, acts)[-1], d126.forward(D, Xf, acts)[-1])[0]
```

ACTUAL OUTPUT

```
after 4,000 steps against N(0, 1): largest |D - D*| where either density is above 0.05: 0.0562; average 0.0163
   x:       -3.0    -2.0    -1.0     0.0     1.0     2.0     3.0
   D*(x):   0.924   0.881   0.182   0.001   0.182   0.881   0.924
   D(x):    0.950   0.845   0.171   0.002   0.208   0.873   0.966
value of the game at D*: V = -0.6537; -log 4 + 2 JSD(p_data, p_g) = -1.3863 + 2 x 0.3663 = -0.6537
the trained discriminator reaches V = -0.6588 on 200,000 fresh samples of each (at most V(D*) = -0.6537)
if the generator matched the data exactly, D* = 0.5 everywhere and V = -log 4 = -1.3863: the generator's best
    possible score
Actual output from this lesson's run.
```

What it means

The trained discriminator follows the formula: within 0.0562 of D* wherever either density has real mass, 0.0163 on average. The value of the game at D* is -0.6537, and –log 4 + 2 JSD gives the same number (JSD = 0.3663). The trained discriminator reaches -0.6588, just below the best possible value, as it must be. A generator that matched the data would bring the value down to –log 4 = -1.3863.

WHY CHOOSE THIS?

Explains what a GAN is aiming at, in a case small enough to check exactly.

WHY NOT THIS? (TRADEOFFS)

Assumes a perfect discriminator; real ones are trained for a few steps at a time.

4. A 1-D game: watching the generator find the data

DEFINITION — start here, no prior knowledge assumed

Both players are trained together on the two-peaked data, five times with different seeds. Every 250 steps the generator's samples are compared with real ones by the 1-D **Wasserstein distance** (sort both samples, pair them up, average the gaps: how far the samples must move to become the data). Two real samples differ by sampling noise alone, which sets a floor.

REAL-WORLD SCENARIO

Learning to throw darts at two bullseyes when a coach only says 'warmer' or 'colder' about each throw.

SPECIFICATION

Players	G [4, 32, 32, 1] (noise of length 4), D [1, 32, 32, 1]; Day 126's Adam, 0.001 for both
Training	batches of 256, 5,000 steps, 5 seeds; one D step, then one G step
Watched	every 250 steps: Wasserstein distance, and the share of fakes above 0 (0.5 if both peaks are covered)

IMPLEMENTATION

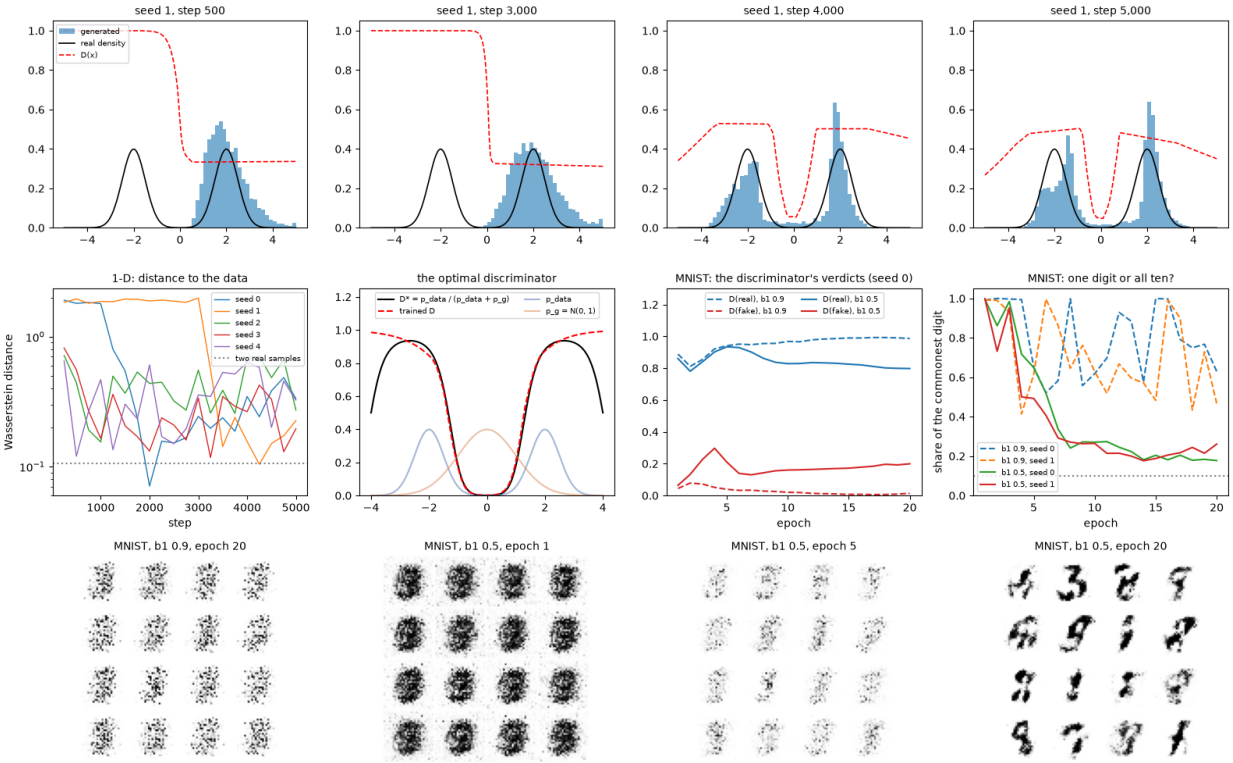
```
def w1(A, B):
    return float(np.mean(np.abs(np.sort(A.ravel()) - np.sort(B.ravel()))))
```

ACTUAL OUTPUT

```
generator [4, 32, 32, 1], discriminator [1, 32, 32, 1], Adam 0.001 for both, batches of 256, 5,000 steps, 5
    seeds
distance to the data: 1-D Wasserstein distance between 5,000 generated and 5,000 real samples; two real
    samples give 0.106
   seed       250     500   1,000   2,000   3,000   4,000   5,000   share of fakes above 0 at each mark
      0     1.904   1.803   1.792   0.071   0.243   0.346   0.333   1.00 1.00 1.00 0.52 0.49 0.49 0.50
      1     1.837   1.950   1.864   1.881   1.983   0.156   0.225   1.00 1.00 1.00 1.00 1.00 0.51 0.51
      2     0.715   0.442   0.154   0.436   0.552   0.692   0.272   0.54 0.46 0.50 0.51 0.50 0.49 0.48
      3     0.816   0.557   0.164   0.132   0.337   0.264   0.196   0.40 0.55 0.50 0.51 0.51 0.51 0.49
      4     0.649   0.120   0.466   0.605   0.356   0.635   0.324   0.51 0.51 0.49 0.48 0.49 0.49 0.48
over the last 2,000 steps the distance moves between 0.104 and 0.709: it keeps changing instead of settling
    (the data's own floor is 0.106)
Actual output from this lesson's run.
```

What it means

Three seeds found both peaks within 1,000 steps. Two did not: seed 0 put every sample on the right peak until somewhere between 1,000 and 2,000 steps (distance 1.792, then 0.071), and seed 1 until between 3,000 and 4,000 (1.983, then 0.156). The figure shows why it can escape: while the fakes sit on one peak, D says 'real' everywhere on the empty peak, so any fake that strays there is rewarded. Once both peaks are covered, nothing settles: over the last 2,000 steps the distance moves between 0.104 and 0.709, sometimes as close as two real samples (0.106), sometimes far off. Unlike a loss going down, a GAN's progress has to be watched during training.



Top: seed 1 at four moments (blue: fakes; black: the real density; red: D(x)). Middle: the distance to the data for every seed; the trained and optimal discriminators; the MNIST discriminator's verdicts; the share of the commonest digit. Bottom: MNIST samples from the same 16 noise vectors.

WHY CHOOSE THIS?

Small enough to see every sample and the discriminator's whole verdict.

WHY NOT THIS? (TRADEOFFS)

One dimension and two peaks; Day 135 uses eight peaks to measure collapse properly.

5. A GAN on MNIST, watched during training

DEFINITION — start here, no prior knowledge assumed

The same parts, on images: G turns 64 noise numbers into 784 pixels (sigmoid outputs), D reads 784 pixels. Generated digits have no labels, so a **judge** (Day 116's softmax regression, trained on real digits only) names the digit in each sample. If the generator makes all ten digits, the judge's labels spread over all ten. The distance from each sample to its nearest training image checks for copying. Adam's **momentum** b1 (how much of the previous gradient each step keeps) is compared at Day 126's 0.9 and at 0.5, the value Radford et al. recommended for GANs in 2015; nothing else changes.

REAL-WORLD SCENARIO

A forger who keeps reusing the one fake that once fooled the inspector, instead of learning to paint.

SPECIFICATION

Players	G [64, 256, 784], D [784, 256, 1]; Adam at 0.0002 for both, b1 = 0.9 or 0.5
Training	the first 20,000 training images, batches of 128, 20 epochs, seeds 0 and 1
Watched	every epoch, on the same 2,000 noise vectors: rarest and commonest digit, judge's confidence, ink, nearest training image

IMPLEMENTATION

```
class AdamGAN(d126.Adam):
    def __init__(self, net, lr=2e-4, b1=0.5):
        super().__init__(net, lr); self.b1 = b1
    def step(self, net, grads):
        self.t += 1; c1, c2 = 1 - self.b1 ** self.t, 1 - 0.999 ** self.t
        for (W, b), (gW, gb), m, v in zip(net, grads, self.m, self.v):
            for p, g, mm, vv in ((W, gW, m[0], v[0]), (b, gb, m[1], v[1])):
                mm *= self.b1; mm += (1 - self.b1) * g; vv *= 0.999; vv += 0.001 * g * g
                p -= self.lr * (mm / c1) / (np.sqrt(vv / c2) + 1e-8)
```

ACTUAL OUTPUT

```
the judge (softmax regression on pixels, never shown a generated image): test accuracy 0.8877, average top
    probability on real digits 0.8995
real test digits: ink (average pixel) 0.133; distance to the nearest of the 20,000 training images 4.49
generator [64, 256, 784] with sigmoid pixels, discriminator [784, 256, 1]; Adam, learning rate 0.0002 for
    both;
20,000 training images, batches of 128, 20 epochs. Each epoch, on 2,000 fixed noise vectors: the judge's share
    of the rarest
and commonest digit (an even spread is 0.100 each), its confidence, ink, and the distance to the nearest
    training image
Adam b1 = 0.9, seed 0:   epoch  D(real)  D(fake)  rarest  commonest  confidence    ink  nearest
                             1    0.888    0.044   0.000      0.992      0.7976  0.256     8.22
                             3    0.854    0.072   0.000      0.997      0.6795  0.201     9.23
                             5    0.941    0.039   0.000      0.651      0.4116  0.160     8.83
                            10    0.968    0.020   0.000      0.619      0.3773  0.148     9.06
                            15    0.989    0.007   0.000      1.000      0.7112  0.142     9.19
                            20    0.986    0.014   0.000      0.630      0.4315  0.085     6.36
Adam b1 = 0.9, seed 1:   epoch  D(real)  D(fake)  rarest  commonest  confidence    ink  nearest
                             1    0.894    0.039   0.000      0.992      0.8499  0.262     7.93
                             3    0.869    0.065   0.000      0.930      0.7173  0.207     8.96
                             5    0.951    0.036   0.000      0.615      0.5725  0.165     8.99
                            10    0.963    0.022   0.000      0.629      0.3663  0.147     9.16
                            15    0.992    0.007   0.000      0.483      0.3249  0.131     8.72
                            20    0.955    0.057   0.000      0.468      0.4471  0.069     5.61
Adam b1 = 0.5, seed 0:   epoch  D(real)  D(fake)  rarest  commonest  confidence    ink  nearest
                             1    0.860    0.064   0.000      0.996      0.8303  0.244     7.17
                             3    0.840    0.226   0.000      0.985      0.6446  0.065     4.76
                             5    0.934    0.210   0.000      0.649      0.5721  0.040     4.02
                            10    0.828    0.160   0.043      0.271      0.6720  0.118     5.69
                            15    0.825    0.172   0.026      0.204      0.7224  0.115     5.74
                            20    0.798    0.199   0.035      0.177      0.7242  0.118     5.74
Adam b1 = 0.5, seed 1:   epoch  D(real)  D(fake)  rarest  commonest  confidence    ink  nearest
                             1    0.855    0.060   0.000      0.999      0.9300  0.254     7.12
                             3    0.849    0.234   0.000      0.953      0.6450  0.060     4.62
                             5    0.935    0.214   0.000      0.492      0.5331  0.038     4.20
                            10    0.831    0.157   0.031      0.265      0.6751  0.117     5.70
                            15    0.825    0.172   0.027      0.188      0.7234  0.118     5.74
                            20    0.800    0.191   0.053      0.261      0.7354  0.120     5.81
(247 s for the four runs)
b1 = 0.9, seed 0, final spread of digits: 0.00 0.00 0.14 0.01 0.00 0.63 0.00 0.19 0.00 0.03
b1 = 0.9, seed 1, final spread of digits: 0.00 0.01 0.00 0.00 0.17 0.47 0.00 0.32 0.00 0.04
b1 = 0.5, seed 0, final spread of digits: 0.03 0.10 0.09 0.13 0.18 0.06 0.07 0.13 0.11 0.10
b1 = 0.5, seed 1, final spread of digits: 0.06 0.06 0.08 0.10 0.26 0.06 0.05 0.13 0.12 0.07
Actual output from this lesson's run.
```

What it means

With b1 = 0.9 the game is lopsided: by epoch 20 D gives fakes 0.014 and 0.057 (the two seeds), and the generator never made some digits at all (rarest share 0.000 at every epoch shown); its commonest digit took 0.630 and 0.468 of its samples. The bottom-left grid shows what that means: 16 different noise vectors, nearly the same speckled image.

With b1 = 0.5, both seeds make every digit by epoch 10 (rarest 0.043 and 0.031), the commonest ends at 0.177 and 0.261, and D settles at about 0.8 on real and 0.2 on fake. The samples have about the right ink (0.118 against 0.133 for real digits) and are not copies: they are 5.74 from their nearest training image, while a real test digit is 4.49 from its nearest. They are still rough: the judge is 0.7242 confident on them, against 0.8995 on real digits.

Why momentum matters here is not tested directly. A likely reason: with 0.9, each step keeps 90% of the previous gradient, so the generator keeps moving towards wherever the discriminator was weak several steps ago, after the discriminator has moved on.

WHY CHOOSE THIS?

A generator that never sees a formula for p(x), only a critic's verdicts.

WHY NOT THIS? (TRADEOFFS)

Sensitive to settings a supervised network barely notices; two seeds per setting is not a guarantee.

Interview preparation — technical questions

Q: What does each player in a GAN minimise?

A: D: –log D(real) – log(1 – D(fake)). G: –log D(G(z)) in practice (the non-saturating form).

Q: How does the generator get a gradient?

A: By backpropagating its loss through D to D's input, then through G; D's weights are held still.

Q: Why the non-saturating loss?

A: When D is sure (D(fake) = 0.0025), log(1 – D) gives 0.0025 per fake and –log D gives 0.9975.

Q: How would you check a GAN implementation?

A: Losses at D = 0.5 (2 log 2 and log 2), a finite-difference check of G's gradient through D, and D* on a known density.

Q: How do you know a GAN on MNIST is not just copying?

A: Compare each sample's distance to its nearest training image with a real test digit's: 5.74 against 4.49 here.

Interview preparation — theoretical questions

Q: What is the optimal discriminator?

A: D*(x) = p_data(x) / (p_data(x) + p_g(x)); trained D matched it to 0.0562 here.

Q: What does a GAN minimise against a perfect discriminator?

A: –log 4 + 2 JSD(p_data, p_g): the Jensen–Shannon divergence.

Q: Why must a GAN be watched during training?

A: Its losses do not fall steadily; the 1-D distance kept moving between 0.104 and 0.709 after both peaks were found.

Q: What is mode collapse?

A: The generator covers only part of the data: here, with b1 = 0.9, some digits were never generated.

Key takeaway and what's next

> **KEY TAKEAWAY**
>
> A GAN learns to generate with no formula for p(x): the generator improves only through the gradient of a discriminator's verdict. Every part was checked by hand, against finite differences and against the formula for the optimal discriminator. In training, the 1-D game found both peaks but never settled, and on MNIST one ordinary setting of Adam was the difference between ten digits and a few near-identical images.

- Gradient through D: worst relative error 4.2e-07; trained D within 0.0562 of D*.
- 1-D game: two of five seeds sat on one peak for 1,000 to 3,000 steps; none settled.
- MNIST, b1 0.9: some digits never made; commonest digit 0.630 of samples.
- MNIST, b1 0.5: all ten digits by epoch 10; samples 5.74 from the nearest training image.

Suggested watch order, if starting from zero

- 1. Ian Goodfellow: Generative Adversarial Networks (NIPS 2016 tutorial) — the inventor's own tutorial: the game, the optimal discriminator and the non-saturating loss.
- 2. Generative Adversarial Networks (GANs) - Explained (DataMListic) — a shorter walk-through of the two players and the minimax objective.

Day 135 preview

- **GAN training instability**: mode collapse, measured on eight known modes, with the minimax loss and the balance between the players.